Music Recommender Systems
Six Methods, One Honest Comparison

Individual Project - ESADE MSc - Recommender Systems (Prof. Marc Torrens)
Dataset: Last.fm HetRec 2011 (implicit feedback)

1. Executive summary

This project builds a music recommender prototype on the Last.fm HetRec 2011 implicit-feedback dataset and compares six methods - three non-personalised popularity baselines, item-item and user-user collaborative filtering, content-based filtering, and matrix factorisation - on a single held-out split using both accuracy and beyond-accuracy metrics. The central finding is that personalisation more than doubles baseline accuracy (item-item CF reaches Precision@10 = 0.175 versus 0.069 for the best popularity baseline), but no single method wins on every objective. Accuracy, diversity, novelty, catalogue coverage, popularity bias, and scalability pull in different directions, so the recommended production design is a portfolio rather than one model.

2. Problem and data

The music track uses implicit feedback only: the signal is how many times each user played each artist, with no explicit ratings and therefore no observed dislikes. This shapes every downstream decision - preference must be inferred as confidence, and evaluation is a ranking problem (did we surface the held-out artists?) rather than rating prediction.

- **Scale:** 1,892 users x 17,632 artists, 92,834 interactions.
- **Sparsity:** 99.72% of the user-artist matrix is empty.
- **Side data:** 11,946 tags with 186,479 (user, artist, tag) assignments, and a user friendship graph (unused here, a possible extension).

3. Exploratory analysis: know your data

Listening is extremely concentrated. The Gini coefficient of plays-per-artist is 0.893; the top 100 artists account for 43.7% of all listening, while the median artist has just a single listener. This long tail is the empirical basis for the project's thesis: a recommender optimised only for accuracy can lean on a tiny set of famous artists and still look respectable, while ignoring almost the entire catalogue.

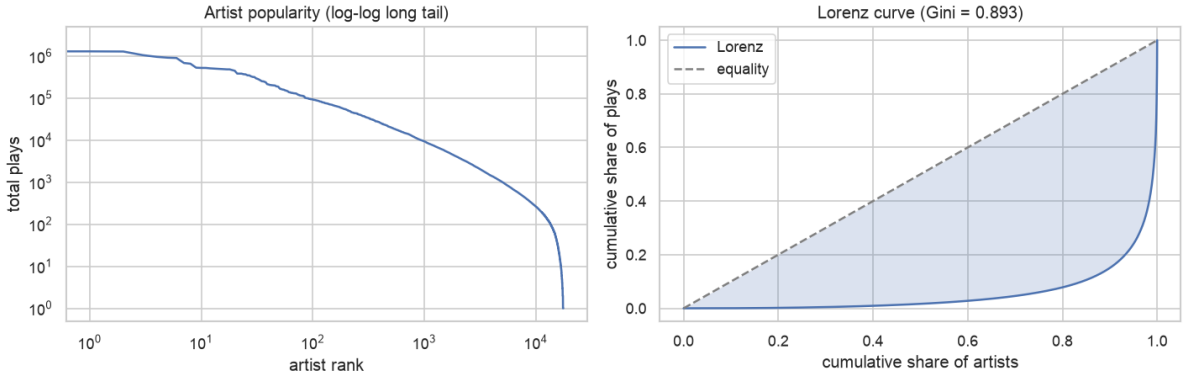

Figure 1. Artist popularity long tail (left) and the Lorenz curve of play concentration (right, Gini = 0.893).

Raw play counts span several orders of magnitude, which motivates log-scaling and confidence weighting before any model sees the data (Figure 2).

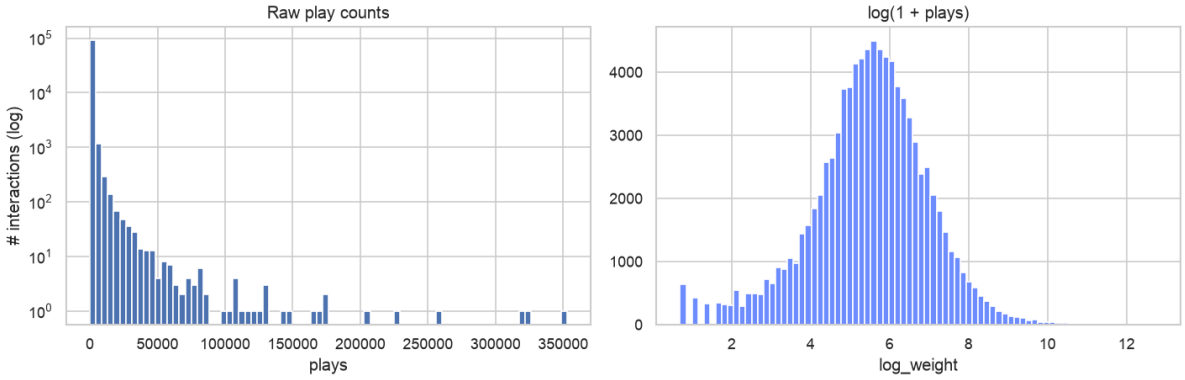

Figure 2. Raw play counts (left) versus log(1 + plays) (right).

4. System design and engineering decisions

The codebase is organised as a reusable **src/recsys** package (data, models, eval, utils) with a thin Flask prototype on top and notebooks reserved for exploration. Several deliberate decisions shaped the work:

- **One model interface.** Every recommender implements the same fit / recommend contract, so the evaluation harness and the app treat all methods interchangeably. This is what makes a fair, apples-to-apples comparison possible.
- **One shared split and harness.** A single per-user leave-out split and a single scoring loop are reused by every method; swapping the model keeps everything else constant.
- **Hybrid implementation.** Core algorithms (cosine kNN, TF-IDF profiles, ALS) are hand-implemented to demonstrate understanding, while numpy / scipy / sklearn provide the heavy linear algebra for speed.
- **Test-driven development.** 75 tests at 94% coverage, including closed-form and known-answer checks for the numerically tricky pieces (metrics, the ALS update).
- **Agile, vertical-slice first.** A thin end-to-end path (load -> popularity -> one screen -> one metric) was built before deepening any module, de-risking integration early.

Implicit feedback is handled with the Hu, Koren and Volinsky (2008) confidence formulation, c = 1 + alpha * log(1 + plays): every observed play is a weak-to-strong vote of confidence, sub-linear in the count so that a few superfans cannot dominate.

5. The six methods and their reasoning

5.1 Popularity baselines (non-personalised)

Three definitions of popular - total plays, distinct listeners, and a damped variant that suppresses mega-hits. These are the comparison floor and the cold-user fallback. Counting distinct listeners beats counting total plays, because total plays let a handful of superfans distort the ranking. Complexity is trivial (a single sorted aggregate).

5.2 Collaborative filtering (item-item and user-user kNN)

Item-item CF scores an artist by its cosine similarity to the artists a user already plays; user-user CF scores by what similar-taste users play. Cosine similarity is hand-implemented as row-normalisation followed by a sparse matrix product, with optional top-k neighbour pruning to bound memory. Item-item is the stronger and more scalable choice under extreme sparsity because artist-to-artist co-occurrence is a more stable signal than user-to-user overlap.

5.3 Content-based filtering (artist tags)

Each artist is represented as a TF-IDF vector over its tags, so distinctive tags outweigh generic ones. A user profile is the play-weighted average of the tag vectors of the artists they play, and recommendations are the nearest artists in tag space. Its structural advantage is cold-start: it can recommend an artist with no listening history at all, which neither CF nor matrix factorisation can do.

5.4 Matrix factorisation (implicit ALS)

Implicit Alternating Least Squares (Hu, Koren and Volinsky 2008) is implemented from scratch. It learns a short latent factor vector per user and per artist whose dot product reconstructs the confidence-weighted preferences. ALS alternates between two closed-form ridge-regression half-steps (fix items, solve users; fix users, solve items), using the YtY + Yt(C-I)Y trick so the shared term is computed once per iteration.

Correctness is verified against a closed-form single-step solution and a synthetic two-cluster dataset, not only against a library.

6. Evaluation methodology

All methods are scored on the same per-user leave-out split (seed 42, 1,884 evaluable users) across three metric families:

- **Accuracy:** Precision@10, Recall@10, MAP@10, NDCG@10.
- **Beyond-accuracy:** catalogue coverage, intra-list diversity (1 - mean pairwise tag cosine), novelty (mean self-information), and popularity bias (mean recommended popularity plus recommendation exposure Gini).
- **Operational:** training time and per-request serving latency.

7. Results

Personalisation clearly works: every personalised method beats every popularity baseline on accuracy, and item-item CF roughly doubles to triples the baseline across ranking metrics (Figure 3). Notably, the simpler item-item CF beats the more complex ALS on this small, dense dataset - a reminder that sophistication is not automatically valuable.

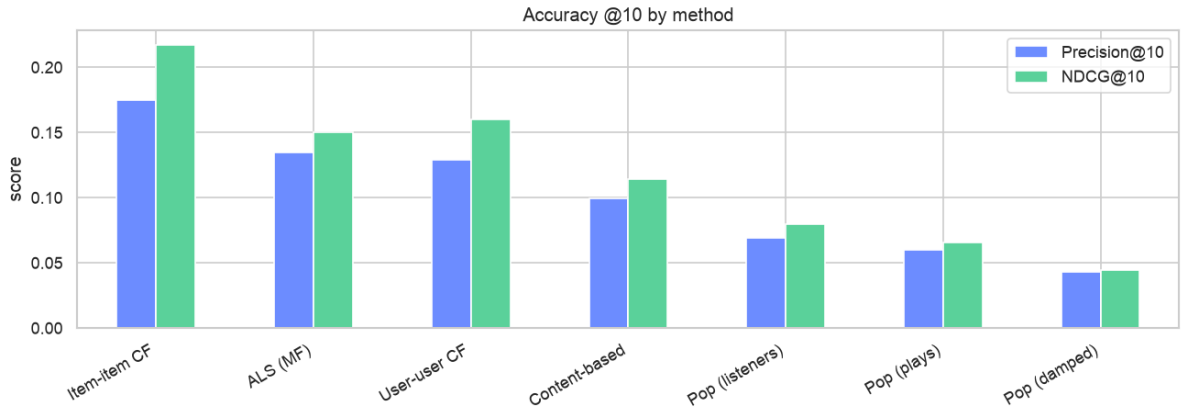

Figure 3. Accuracy at k=10 by method.

But accuracy is not the whole story. Figure 4 shows that no method wins on coverage, diversity, and novelty simultaneously, and Figure 5 makes the accuracy-versus-diversity tension explicit.

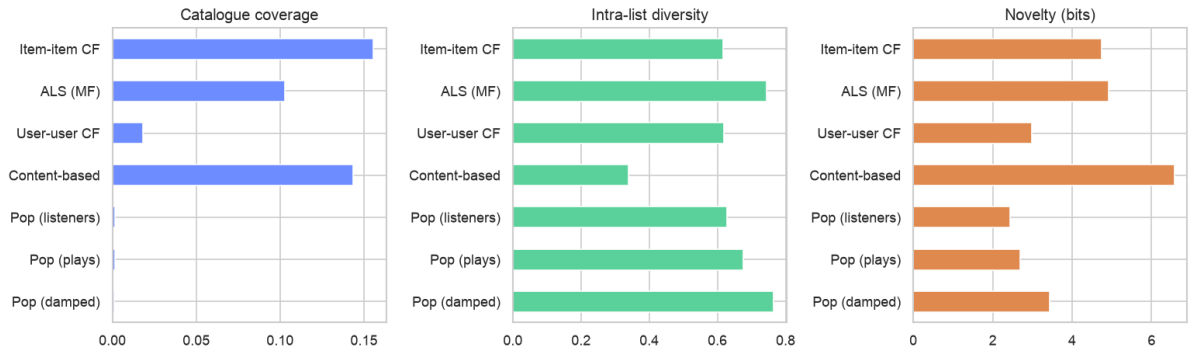

Figure 4. Beyond-accuracy metrics: coverage, diversity, novelty.

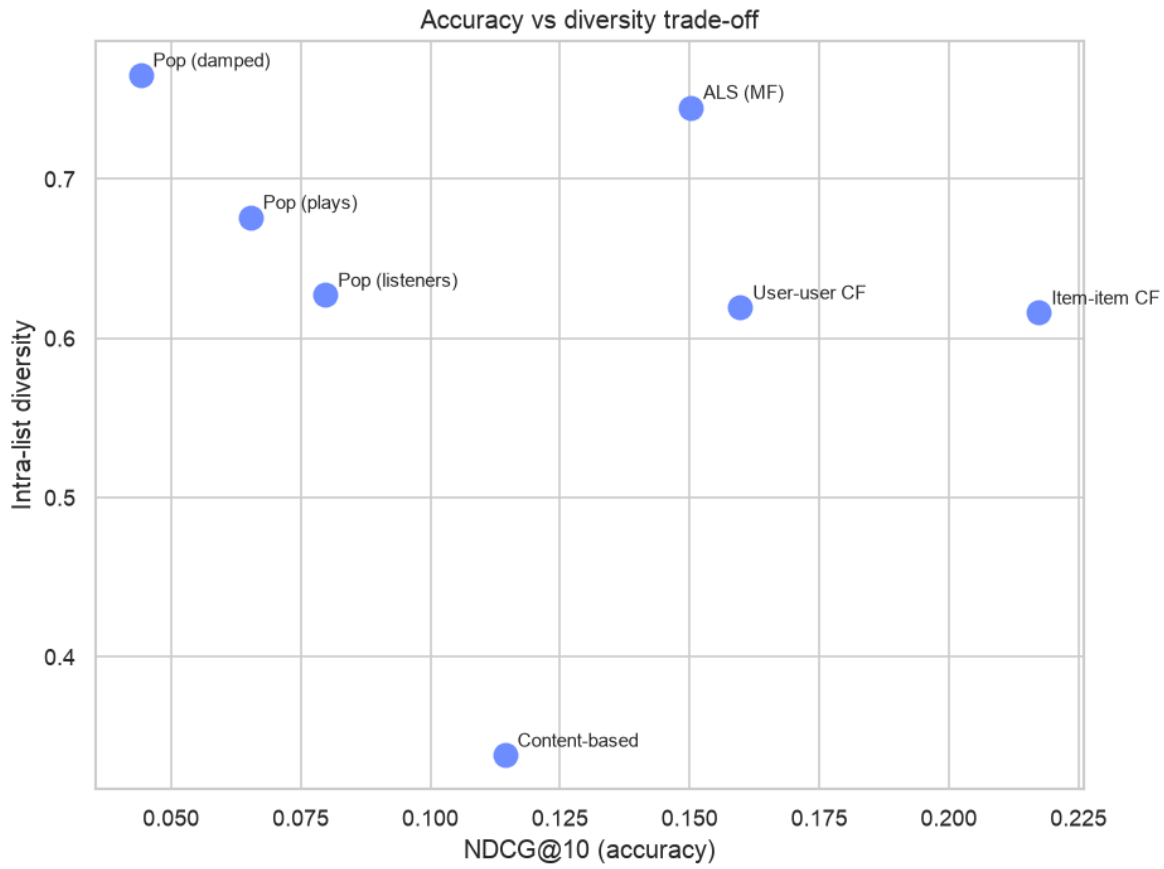

Figure 5. The accuracy versus diversity trade-off.

Popularity bias is measurable and large. Popularity recommenders show the same handful of artists to everyone (catalogue coverage near 0.001 and the highest exposure Gini), while content-based filtering is the least biased toward popular artists. Even user-user CF, though personalised, leans noticeably on crowd-pleasers. Training cost and serving cost also differ sharply by method family (Figure 6).

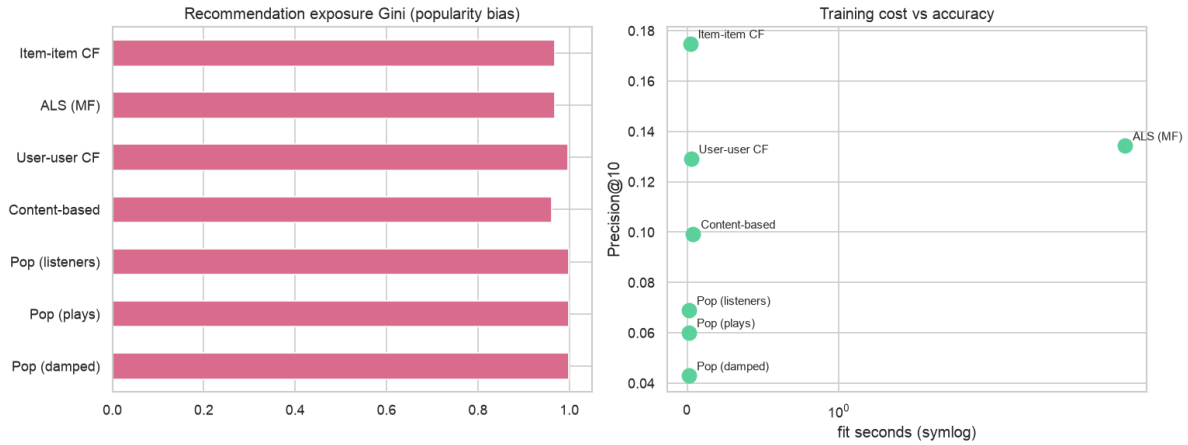

Figure 6. Popularity bias via exposure Gini (left) and training cost versus accuracy (right).

Method	P@10	NDCG	Cov	Div	Nov	PopBias	fit s
Item-item CF	0.175	0.217	0.156	0.616	4.75	0.084	0.02
ALS (MF)	0.134	0.150	0.103	0.745	4.91	0.053	6.27
User-user CF	0.129	0.160	0.018	0.619	2.98	0.140	0.03
Content-based	0.099	0.114	0.143	0.338	6.57	0.034	0.04
Pop (listeners)	0.069	0.080	0.002	0.627	2.44	0.186	0.01
Pop (plays)	0.060	0.065	0.001	0.676	2.68	0.165	0.01
Pop (damped)	0.043	0.044	0.001	0.765	3.44	0.120	0.01

Table 1. All methods on the same held-out split (k=10, seed 42, 1,884 users). Cov = catalogue coverage, Div = intra-list diversity, Nov = novelty (bits), PopBias = mean popularity of recommended artists.

8. Critical analysis

- **No universal winner.** Item-item CF leads accuracy and coverage; ALS gives the most diverse of the accurate lists and the lowest popularity bias among strong methods; content-based is the most novel but the least diverse (a tag filter bubble); popularity is a cheap floor that is heavily biased and repetitive.
- **Bias hides inside 'personalised'.** User-user CF has higher popularity bias (0.140) than item-item CF (0.084); personalised does not automatically mean fair.
- **Cold-start.** Only content-based can recommend an artist with no listening history, a real advantage despite its lower accuracy.
- **Scalability is a trade, not a winner.** Memory-based CF trains in about 0.02s but stores an item-item similarity matrix that grows with the catalogue squared; ALS pays about 6.3s of training but serves from compact factors in about 0.2ms.
- **Honesty about metrics.** Exposure Gini is high for every method because ten slots across 1,884 users can only ever touch a fraction of 17,632 artists; it is read as a relative concentration measure, and intra-list diversity is only meaningful in tag space.

9. Honest limitations and threats to validity

The results above should be read with the following limitations in mind. They are stated explicitly rather than buried, because several of them would change the numbers if addressed.

- **Random, not temporal, split.** Evaluation uses a per-user random hold-out, which can leak later listens into training. A time-based split would be more realistic and would likely lower the reported scores.
- **Single split, no significance testing.** Results are one seed with no cross-validation or confidence intervals, so small gaps between methods (for example user-user CF versus ALS) are not statistically validated.
- **Light hyperparameter tuning.** Only ALS was tuned, and only coarsely; its factors and iterations were capped for runtime, so it is likely under-fit. The planned cross-check against the reference implicit library was not done, so the hand-written ALS is validated only by unit tests.
- **Accuracy rewards re-discovery, not discovery.** Relevant items are artists the user already played and we held out, so accuracy metrics favour re-finding known tastes and are structurally biased toward popular items; beyond-accuracy metrics soften but do not remove this.
- **Cold-start is argued, not measured.** Content-based filtering's cold-start advantage is structural but was never quantified with a held-out new-user or new-artist experiment.
- **Beyond-accuracy blind spots.** Intra-list diversity is computed only in tag space (untagged artists are skipped), novelty is derived from training popularity, and exposure Gini is near-saturated for every method because ten slots across 17,632 artists can only ever touch a fraction of the catalogue.
- **Small, dense dataset.** HetRec keeps roughly the top 50 artists per user, which makes the popularity baseline unusually strong and may flatter the neighbourhood methods; generalisation to a larger, sparser catalogue is untested.
- **Scalability is descriptive only.** Training and serving costs are reported at this dataset's scale; none of the methods were stress-tested at production scale.
- **Unused signal.** The user friendship graph is available but not exploited, leaving a social recommendation angle on the table.
- **Prototype UX.** The interface is functional and not user-tested.

10. Conclusion and recommendation

Accuracy is necessary but not sufficient. The right production system for this dataset is a portfolio: ship item-item CF as the default for its strong accuracy, broad coverage, and near-instant training; blend in content-based filtering for cold-start coverage and novelty; and keep a popularity model as the fallback for brand-new users. Natural next steps are hybrid score blending, a learning-to-rank layer, a temporal rather than random split, and a cross-check of the hand-written ALS against the implicit library.

Reproducibility. The dataset is fetched by scripts/download_data.py; scripts/build_processed.py caches the processed artifacts; scripts/evaluate_baselines.py regenerates the comparison table; the EDA and evaluation notebooks regenerate every figure; and the full test suite (75 tests, 94% coverage) guards correctness.